

GUESTGUARD

Inspector Quiz, API, and D1 Developer Guide

How question selection, attempts, grading, authentication, and persistence actually work.

Codebase snapshot: August 4, 2026

Short version: D1 does not grade anything by itself. The Cloudflare Worker grades in memory, then writes the attempt and result to D1. The certification answer key never ships with the public training site.

Audience: the next developer who has to change a question, debug an attempt, rotate an API URL, or figure out why a learner cannot retake the exam.

Contents

1. The moving parts
2. End-to-end request flow
3. D1 schema and row lifecycle
4. Question bank and selection
5. Grading and retakes
6. Authentication and API boundaries
7. Function map
8. Safe edit recipes
9. Local development and deployment
10. Debugging and operational queries
11. Tester shortcuts
12. Gotchas and change checklist

Heads up: This guide describes the repository as it exists now. The certification bank has 101 source questions; each initial exam serves 50.

1. The moving parts

There are three systems in play. Keeping them separate will save you a lot of confusion.

Training frontend	module5.html, module5.js, quiz-config.js	Renders questions, keeps an answer draft, sends authenticated API requests, and shows the result. It never receives canonical answer IDs.
Quiz Worker	quiz-worker/src/index.js	Authenticates the learner, selects questions, shuffles display IDs, grades submissions, enforces attempts, and writes D1.
D1	inspector-quiz-db	Stores quiz metadata, immutable attempt plans, submitted answers, scores, pass/fail state, and append-only result rows.
Question source	quiz-worker/src/question-bank.js	The 101 canonical certification questions and correct option IDs. This file is bundled into the Worker, not uploaded as a public site asset.

The certification quiz row in quizzes is mostly metadata: seed, title, and pass mark. Its JSON question and answer fields are empty placeholders because `quizBank()` switches that seed to the code-based question bank.

Heads up: Editing the D1 questions JSON will not change the certification exam while the seed is `inspector-certification-v1`. Edit `question-bank.js` and redeploy the Worker.

Repository boundary

- The root Worker serves static training assets.
- The `quiz-worker` directory is excluded by `.assetsignore`, so the answer key and server code are not public training assets.
- R2 hosts video files only. It has no quiz or D1 binding.

2. End-to-end request flow

Starting an attempt

```
Browser Quiz Worker D1
POST /attempts/start -> validate Bearer token
{ quizSeed } hash issuer + subject
load quizzes row -> SELECT quiz metadata
find active attempt -> SELECT active
or load history -> SELECT submitted history
select + shuffle 50
INSERT OR IGNORE attempt -> quiz_attempts
<- public question text, shuffled display IDs, attemptId
```

Submitting an attempt

```
Browser Quiz Worker D1
POST /attempts/submit -> authenticate same learner
{ attemptId, answers } load owned attempt -> SELECT by id + learner_id
map display IDs to canonical IDs
compare with server answer key
claim active row atomically -> UPDATE ... status = active
append audit/result row -> INSERT results
<- score, total, rounded percent, pass/fail, missed question text
```

The display IDs are deliberately temporary. A browser sees IDs like q7 and q7o3. D1 stores a question plan mapping those IDs back to canonical IDs such as q032 and o4. That lets the Worker grade accurately without leaking the answer key in the start response.

3. D1 schema and row lifecycle

table	important columns	purpose
quizzes	seed PK, title, questions, answer_key, pass_mark	Quiz configuration lookup. Certification questions come from Worker code despite the JSON columns.
quiz_attempts	id PK, learner_id, attempt_number, question_plan, status, score, passed, incorrect_ids, answers, timestamps	Source of truth for active and submitted attempts. One row per learner/quiz/attempt number.
results	id, seed, user_id, score, total, passed, answers, submitted_at	Simple append-only result record used for reporting or auditing.

Attempt state machine

```
new request -> active -> submitted
\-> never reopened
```

```
A passed history -> terminal complete response
Four submitted attempts without a pass -> terminal exhausted response
```

The unique index

quiz_attempts_learner_number_idx covers (learner_id, quiz_seed, attempt_number). It is not optional. Two tabs can calculate the same next attempt number at the same time. INSERT OR IGNORE plus that index makes one row win; both callers then receive the canonical active attempt instead of creating duplicates.

Submission claim

The final update includes WHERE status = 'active'. The Worker checks meta.changes === 1. If two tabs submit together, only one claims the row; the other gets HTTP 409. This prevents double results and double grading.

4. Question bank and selection

Certification bank layout

Category	Bank size	Initial quota
GuestGuard purpose and inspector role	10	5
Workflow, remediation, and documentation	10	5
Exterior, access, pools, and pets	10	5
Fire safety and emergency evacuation	11	5
Electrical, utilities, detectors, and hazards	10	5
Accessibility measurements and judgment	10	5
Kitchen and appliances	10	5
Bathroom and hallway	10	5
Bedroom and common room	10	5
Overall property and guest experience	10	5

Initial attempt

`selectQuestions()` takes exactly five questions from each of ten categories, producing 50 questions. It independently shuffles each category selection and then shuffles the combined set.

Retakes

A retake size is twice the number missed on the previous attempt, capped at the bank size. Half of the slots target the same categories as the missed questions. The rest prefer questions the learner has not seen, then fall back to previously seen questions without duplicates.

Heads up: A perfect or passing submission never creates a retake. A failed submission with zero missed questions is logically impossible, so `selectQuestions()` throws if asked to build one.

5. Grading and retakes

What actually grades

The Worker grades. D1 persists the outcome. The grading loop walks the stored question plan, finds the selected display option, maps it to the canonical option, and compares it with the canonical answer key from `question-bank.js`.

```
rawPercent = (score / questionCount) * 100
displayedPercent = Math.round(rawPercent)
passed = rawPercent >= Number(quiz.pass_mark)
```

The raw percentage decides pass/fail; rounding is display-only. This matters on short retakes. A raw 79.6 does not become a pass just because the UI displays 80.

	Value	Location
Initial question count	50	INITIAL_QUESTION_COUNT in <code>quiz-worker/src/index.js</code>
Maximum attempts	4 total (initial + 3 retakes)	MAX_ATTEMPTS in <code>quiz-worker/src/index.js</code>
Pass mark	80 percent	<code>quizzes.pass_mark</code> in D1; seeded as 80 in <code>schema/migration</code>
Certification bank	101 canonical questions	<code>quiz-worker/src/question-bank.js</code>

What the response reveals

- Score, total, rounded percent, pass mark, remaining retakes, and whether another retake is allowed.
- Missed question number and text.
- No correct answer text and no canonical option IDs.

6. Authentication and API boundaries

Browser token

The portal redirects into training with a short-lived token. `training-api.js` exchanges it, stores the access token in browser `localStorage`, and `module5.js` sends it as `Authorization: Bearer ....`

Worker validation

`authenticatedLearner()` forwards the bearer token to `AUTH_VALIDATION_URL`. Only a successful validation response is accepted. It then decodes the already-validated JWT, takes a stable subject claim, combines issuer and subject, and SHA-256 hashes them. D1 receives `portal:<hash>`, not an email address or raw token.

No bearer token	401 Authentication required
Expired/invalid token	401 Invalid or expired authentication
Validation service unreachable	503 Authentication service unavailable
Attempt belongs to another learner	404 Unknown attempt
Second submission	409 already submitted
Origin not allowlisted	No Access-Control-Allow-Origin header

Heads up: The Worker currently validates against the URL in `quiz-worker/wrangler.jsonc`. Changing environments means changing `AUTH_VALIDATION_URL` and the allowed training origin, then redeploying. Do not casually point a test Worker at production identity services.

7. Function map

Function	Role
fetch(request, env)	Routes OPTIONS, /attempts/start, and /attempts/submit. Everything else is 404.
startAttempt(...)	Loads metadata/history, enforces terminal states, selects and inserts the next attempt, resumes an active attempt.
submitAttempt(...)	Validates ownership/payload, grades, atomically submits, inserts results, returns safe feedback.
selectQuestions(...)	Balanced first attempt or targeted retake selection. Exported for direct testing.
buildPlan(...)	Assigns temporary question/option display IDs and shuffles options.
publicQuestions(...)	Removes canonical IDs and answer data from the browser response.
attemptResponse(...)	Rebuilds the public payload when resuming an active attempt.
categoryCoverage(...)	Returns category counts for diagnostics/UI metadata.
quizBank(...)	Chooses the code bank, demo bank, or D1 JSON bank based on seed.
seededRandom(...) / shuffle(...)	Repeatable selection/shuffle helpers using the per-attempt variant seed.
authenticatedLearner(...)	Validates bearer auth and derives the hashed learner ID.
corsHeaders(...) / json(...)	CORS allowlist and hardened JSON response headers.

8. Safe edit recipes

Change or add a certification question

```
// quiz-worker/src/question-bank.js
question('q102', 'fire', 'Question text?', [
  'Wrong answer',
  'Correct answer',
  'Wrong answer',
  'Wrong answer'
], 1) // zero-based correct option index
```

- Use a permanent, unique canonical ID. Never renumber old IDs after attempts exist.
- Use one existing category ID from CURRICULUM_CATEGORIES.
- The final number is zero-based: 0 means first option, 3 means fourth.
- Run the bank invariant check and Worker dry-run, then deploy the quiz Worker. A static-site deploy alone does not update the bank.

Change a question safely

Text edits are fine. Changing an answer after active attempts exist is trickier: their plan stores canonical IDs, but grading uses the current answer key. An in-flight attempt could be graded against the new answer. For a major correction, use a new quiz seed or coordinate an attempt cleanup rather than silently flipping the key.

Change category quotas

Edit testCount values and keep their sum equal to INITIAL_QUESTION_COUNT. Each category needs at least its quota. The Worker throws if either invariant fails.

Change pass mark

Update the D1 quizzes.pass_mark row. The frontend config also says 80, but the server response is authoritative. Keep the frontend copy aligned so labels and assumptions stay honest.

Do not do this

- Do not put the answer key in module5.js or any root static asset.
- Do not delete or reuse canonical question IDs.
- Do not remove the unique attempt index.
- Do not trust learnerId, score, pass/fail, or question IDs supplied by the browser.

9. Local development and deployment

Local setup

```
cd quiz-worker
npm install
npm run db:local
npm run dev

# In another terminal, serve the repository root on localhost:8000
```

The frontend automatically uses `http://localhost:8787` for the quiz API on localhost. Production uses the URL in `quiz-config.js`.

Production checks before deploy

```
node --check src/index.js
node --check src/question-bank.js
npx wrangler deploy --dry-run
npx wrangler whoami
```

Deploy

```
cd quiz-worker
npx wrangler deploy
```

The quiz Worker and training site are separate deployments. Deploying the root static site does not update Worker grading code. Deploying `quiz-worker` does not update HTML, CSS, videos, or tester tools.

Schema changes

Use a new numbered migration for production schema changes. Keep migrations additive where possible. Applying `schema.sql` is fine for a fresh local database, but it is not a substitute for a reviewed production migration.

Heads up: Before any remote write, run `wrangler whoami` and verify Brian's Cloudflare account ID `80b79e71262a5823d6536012847fc8fc`. The quiz Worker config is pinned to that account.

10. Debugging and operational queries

Read-only health queries

```
SELECT seed, title, pass_mark FROM quizzes;

SELECT status, COUNT(*) AS count
FROM quiz_attempts GROUP BY status;

SELECT learner_id, quiz_seed, attempt_number, status, score, total, passed,
datetime(created_at / 1000, 'unixepoch') AS created_utc
FROM quiz_attempts
ORDER BY created_at DESC LIMIT 25;

SELECT name, sql FROM sqlite_master
WHERE type = 'index' AND tbl_name = 'quiz_attempts';
```

Common symptoms

Symptom	Root cause and resolution
401 on start	Missing/expired access token or validation URL mismatch. Check browser auth warning and Worker validation endpoint.
403-looking browser CORS failure	Training origin missing from ALLOWED_ORIGINS. Inspect OPTIONS response.
Active attempt keeps returning	Expected resume behavior. Find the active quiz_attempts row for that hashed learner.
409 on submit	Same attempt already submitted, often another tab. Treat as protection, not random failure.
Question edit not visible	Quiz Worker was not redeployed, or an existing active attempt is being resumed with its old plan.
Learner passed but course not complete	Quiz result saved, but portal training progress POST failed afterward. Quiz D1 and portal progress are separate systems.

Heads up: Never paste access tokens, answer JSON, or full learner hashes into tickets or chat. Query the minimum fields needed and redact identifiers.

11. Tester shortcuts

Tester mode is for the training UI, not D1 administration. Start on any training URL with `?tester=1`, then press the master chord. The eligibility and flags live in `sessionStorage` and disappear when that browser session ends.

Shortcut	Action
Ctrl + Alt + Shift + D	Enable or disable tester mode. Disabling also clears skip and navigation bypass flags.
Ctrl + Alt + Shift + S	Toggle unrestricted video timeline seeking.
Ctrl + Alt + Shift + U	Toggle chapter and exam navigation locks.
Ctrl + Alt + Shift + R	Confirm and clear browser-local training progress only, then reload.
Ctrl + Alt + Shift + H	Show or hide the shortcut card.

What tester mode does not do

- It does not reset `quiz_attempts` or results.
- It does not erase portal-side progress.
- It does not reveal correct answers or bypass Worker grading.
- It does not create authentication. The quiz API still requires a valid bearer token.

Heads up: Use a dedicated test account. If a signed-in tester enables video skipping and lets a video reach ended, normal progress synchronization can record that lesson as complete for that test account.

12. Gotchas and change checklist

Stuff that is easy to miss

- Certification questions are code-backed; the D1 JSON fields are not the live bank for inspector-certification-v1.
- An active attempt freezes question and option display IDs, but the current canonical answer key is used at submission time.
- A result row is inserted only after the attempt row is atomically claimed as submitted.
- The displayed percent is rounded; pass/fail uses the raw percentage.
- Question text shown in missed feedback is safe; correct answers stay server-side.
- Local answer drafts are keyed by attempt ID and are deleted after a successful submission.
- Portal course progress and quiz D1 results are separate writes. One can succeed while the other fails.

Before merging or deploying a quiz change

Canonical question IDs are unique and stable	[]
Every correct option ID exists exactly once	[]
Every category has at least its quota	[]
Category quotas total 50	[]
Pass mark matches the intended policy	[]
No answer key entered a public root asset	[]
Start/resume behavior tested	[]
Concurrent start and duplicate submit tested	[]
401, 404, and 409 paths tested	[]
D1 index still exists	[]
Correct Cloudflare account verified	[]
Worker dry-run and production smoke check passed	[]

If you remember only one thing: the browser asks, the Worker decides, and D1 remembers.